25. İlişkisel (SQL) vs. İlişkisel Olmayan (NoSQL): Hangisi Ne Zaman?

PostgreSQL mi MongoDB mi? Ezbere Seçimlerden Kurtulma Kılavuzu
Bölüm 5 • Zorluk Derecesi: Orta Seviye • Tahmini Okuma Süresi: 12 dk

---

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Yeni başlayan geliştiriciler neden her projeye hemen MongoDB açmaya çalışır ve finansal işlemlerde NoSQL kullanmak nasıl bir felakettir?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 SQL ilişkisel bütünlüğü (ACID) ve katı şemaları korurken; NoSQL esnek şemalar ve yatay ölçeklenme için tasarlanmıştır. - 📌 Eğer verileriniz arasında ilişkiler (Kullanıcı -> Sipariş -> Fatura) varsa, varsayılan tercihiniz her zaman PostgreSQL olmalıdır. - 📌 Modern PostgreSQL, JSONB desteği sayesinde NoSQL'in esnekliğini SQL'in güvenilirliğiyle birleştirir.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir banka kasasını ve bir oyuncak sepetini düşünün.

Banka kasasında (SQL) her kuruşun yeri bellidir. Kim kime ne kadar para gönderdiyse, Ahmet'in hesabından düşüldüğü saniyede Mehmet'in hesabına eklenmelidir. İkisi aynı anda gerçekleşmezse işlem iptal edilir (ACID).

Oyuncak sepetinde (NoSQL) ise kural yoktur! Pelüş ayı, lego parçası, uzaktan kumandalı araba hepsini aynı sepete atabilirsiniz. Hızlıca eşya atmak veya sepeti kucaklayıp başka bir odaya taşımak (yatay ölçeklenme) çok kolaydır.

Eğer banka parasını oyuncak sepetine koyarsanız, paralar kaybolur! Ancak IoT sensörlerinden saniyede 10.000 log topluyorsanız sepet tam aradığınız şeydir.

🧠 2. AI Çağında Yeni Zihniyet Modeli

SQL vs NoSQL Karşılaştırma Rehberi:

```
[İhtiyaç Analizi]
       |
       ├── Veri ilişkisel mi? (Tablolar arası Foreign Key var mı?)
       |   └──> SQL (PostgreSQL, MySQL)
       |
       ├── Finansal işlem veya katı tutarlılık (ACID) şart mı?
       |   └──> SQL (PostgreSQL)
       |
       ├── Veri şeması sürekli değişen dökümanlar mı? (CMS, Dinamik Formlar)
       |   └──> NoSQL (MongoDB, CouchDB) VEYA Postgres JSONB
       |
       └── Milyonlarca geçici oturum veya cache mi?
           └──> Key-Value NoSQL (Redis)
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Baş Veritabanı Mimarı
Yeni bir SaaS platformu kuruyoruz. Sistemde kullanıcılar, faturalar, abonelik
planları ve kullanıcıların yüklediği dinamik anket formları olacak.

## GÖREV:
1. Bu sistem için PostgreSQL mi yoksa MongoDB mi seçmeliyiz? Trade-off (ödünleşim)
analizi yap.
2. Neden PostgreSQL + JSONB kombinasyonunun her iki dünyanın en iyi yönlerini
birleştirdiğini açıkla.
3. Örnek bir kullanıcı ve anket şemasını SQL ve JSONB ile tasarla.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ **Acemi / Kontrolsüz AI Çıktısı (Riskli):**

```
-- Kötü NoSQL Yaklaşımı: İlişkisel veriyi MongoDB'de iç içe gömmek (Denormalization Kabusu)
-- Kullanıcı bir adresini güncellediğinde, 50 farklı sipariş dökümanındaki gömülü adresi elle güncellemek gerekir!
{
  "_id": "order_123",
  "user": { "id": "u1", "name": "Ali", "address": "Eski Adres" },
  "total": 450.00
}
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
-- PostgreSQL ile Hibrit Mükemmellik: Katı İlişkiler + Esnek JSONB
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email VARCHAR(255) UNIQUE NOT NULL,
    created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE invoices (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE RESTRICT,
    amount NUMERIC(10, 2) NOT NULL,
    status VARCHAR(50) DEFAULT 'PENDING',
    created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
);

-- Dinamik ve Şemasız Veri İçin JSONB Kolonu
CREATE TABLE survey_responses (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(id),
    -- Anket soruları ve yanıtları dinamiktir; şema zorunluluğu yok!
    answers JSONB NOT NULL,
    created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
);

-- JSONB Alanı Üzerinde GIN İndeksi ile Işık Hızında Arama
CREATE INDEX idx_survey_answers ON survey_responses USING GIN (answers);
```

💡 **Kod Analizi:** PostgreSQL sayesinde hem finansal fatura verisi `REFERENCES users(id)` ile katı ilişkisel güvencede tutulur, hem de dinamik anket yanıtları `JSONB` kolonunda esnekçe saklanıp `GIN` indeksiyle aranabilir.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Daha proje başlamadan ‘MongoDB daha modern’ deyip ilişkisel sistemi NoSQL’e taşımak.

  - **Çözüm:** Aksi için çok güçlü bir sebep yoksa varsayılan olarak her zaman PostgreSQL ile başlayın.
- **Tuzak:** NoSQL'de ilişkisel tabloları simüle etmeye çalışıp elle `populate` veya `lookup` yazmak.
  - **Çözüm:** Eğer `JOIN` yapmaya ihtiyaç duyuyorsanız yanlış veritabanını seçmişsiniz demektir; SQL'e geçin.
- **Tuzak:** SQL veritabanında para tutarlarını FLOAT olarak saklamak.
  - **Çözüm:** Para tutarları için daima `NUMERIC` veya `DECIMAL` tipi kullanın.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 25: SQL vs NoSQL - Veritabanı Seçim Rehberi*

- 🖥️ **Slayt 1:** SQL ve NoSQL arasındaki felsefe farkı: ACID vs BASE.
- 🖥️ **Slayt 2:** İlişkisel Bütünlük (Foreign Key) neden hayat kurtarır?
- 🖥️ **Slayt 3:** PostgreSQL JSONB: NoSQL esnekliğini SQL'de kullanmak.
- 🖥️ **Slayt 4:** Canlı SQL: GIN indeksi ile JSON içinde sorgu çalıştırmak.

Canlı Kodlama / Ekran Kaydı Senaryosu:
PostgreSQL veritabanında JSONB kolonuna sahip bir tablo açın. `WHERE answers @> '{"rating": 5}'` sorgusunu çalıştırarak JSON içindeki bir alanın milisaniye altında nasıl filtrelendiğini gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenizdeki veritabanı şemasını açın. Eğer NoSQL kullanıyorsanız tablolar arası ilişki kurup kurmadığınızı; SQL kullanıyorsanız gereksiz tablolar yerine JSONB kullanılabilecek alanları analiz edin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Veritabanlarında 'ACID' kısaltmasının 'A' harfi (Atomicity) neyi garanti eder? - A) Veritabanının atom bombasına dayanıklı olduğunu - B) Bir transaction içindeki tüm adımların ya tamamen başarılı olacağını ya da tek bir hata durumunda tüm işlemlerin geri alınacağını (ya hep ya hiç) - C) Sadece Python ile çalıştığını - D) Tabloların otomatik silineceğini > **Doğru Yanıt:** B > **Açıklama:** Atomicity (Bütünlük), ilişkili işlemlerin bölünemez bir bütün olduğunu; başarısızlık durumunda veritabanının önceki temiz durumuna geri döndürüleceğini garanti eder.

Soru 2: PostgreSQL'in `JSONB` kolon tipi ile standart `JSON` tipi arasındaki temel fark nedir? - A) Hiçbir farkı yoktur - B) JSONB veriyi ikili (binary) formatta saklar ve GIN indeksleri ile alan bazlı hızlı arama yapılmasını sağlar - C) JSONB sadece sayılar saklayabilir - D) JSONB daha yavaş çalışır > **Doğru**

Yanıt: B > **Açıklama:** JSONB, JSON verisini parçalanmış ikili formatta tutar ve indeksleme desteği sayesinde devasa JSON belgeleri içinde anında filtreleme imkanı sunar.